Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Genel Konular

- Dersin kaynakları
  - Slayttaki kaynaklar (ekrandan, 00:33:00): Khalid Sayood, *Introduction to Data Compression*; J. Gibson, T. Berger, T. Lookabaugh, *Digital Compression for Multimedia*; D. Salomon, *Data Compression*.
  - Ders notları ağırlıklı olarak Salomon'un kitabından hazırlanmıştır; Sayood'un kitabından da yararlanılmıştır, ikinci kaynak pek kullanılmamıştır. Derste konuların temel taşları anlatılır.
- Dersin konuları (ekrandan, 00:40:00)
  - Veri sıkıştırma nedir ve sıkıştırmada kullanılan terminoloji.
  - Hata bulma ve düzeltme: gönderilen veride bozulma olduğunu anlamak, düzeltebilmek ve kaç bit düzeltilebildiği; verinin arkasına eklenen kontrol bilgisi.
  - Temel teknikler: sezgisel yöntemler, Run-Length Encoding (RLE), Move-To-Front. Bunlar sıkıştırmanın ilk yöntemleridir ve ileride görülecek yöntemlerin içinde ara adım olarak da kullanılır. Örneğin JPEG'in içinde RLE de kullanılır.
  - Statik kodlar: prefix kodlar ve Golomb kodu (slaytta "Colomb"). Statik kodlar varsa dinamik kodlar da vardır.
  - İstatistiksel yöntemler: Shannon-Fano, Huffman ve türevleri, MNP (Microcom Network Protocol) ve türevleri, aritmetik kodlama, PPM. Dokümanda karakterlerin, görüntüde renklerin kullanım sıklığına ve olasılığına bakarlar.
  - Sözlüksel yöntemler (iki kişinin soyadından oluşan yöntem, yani LZ ailesi) ve görüntü sıkıştırma.
  - Ses sıkıştırma kaynak kitaplarda vardır ama dersin içeriğinde yoktur; sesle çalışmış olanlar proje ve seminerlerini bu alana kaydırabilir.
- Sezgisel (ad hoc) yöntemler
  - Belirli bir veriye özgü geliştirilmiş yöntemlerdir. Yalnız cep telefonu numaralarından oluşan bir veri ile gazete makalelerinden oluşan bir veri aynı yöntemle sıkıştırılırsa başarıları farklı olur. Bir girdiye özel geliştirilen yöntem o girdide yüksek, başka girdilerde daha düşük başarı verir.
- Veri sıkıştırma nedir? (ekrandan, 00:50:00)
  - Bir input data stream'in (source stream, original raw data) yerden ve zamandan kazanmak amacıyla boyutu küçültülerek başka bir data stream'e (output, compressed) çevrilmesidir.
  - Sıkıştırılan veri kullanılmak istendiğinde yeniden orijinal hâline getirilir. Hoca bunu, çantaya eşyaları bastırarak tıkıştırmaktan ayırdı: orada içerik aynı kalır, sıkıştırmada ise içeriğin boyutu küçültülür.
- Sabit ve değişken uzunluklu kod
  - Fixed size code: tüm sembollerin aynı bit uzunluğunda kodlanması. Örnek: ASCII, her karakter 8 bit; Türkçede sık geçen "a" da, nadir geçen "j" ve "w" da 8 bittir.
  - Variable size code: sembollerin farklı bit uzunluklarıyla kodlanması; kullanım sayısı fazla olan semboller daha az, az olanlar daha çok bitle kodlanır. Örnek: "a" 3 bit, "j" 10 bit. Bit uzunluklarını kişi seçmez, algoritma belirler.
- Artıklık (redundancy)
  - Sıkıştırılmış veri içindeki fazlalıktır.
  - Hoca bunu kâğıt katlamayla anlattı: kâğıt hâlâ katlanabiliyorsa artıklığı fazladır; bir noktadan sonra daha fazla katlanamaz.

  - Entropiyle bir sembolün ortalama kaç bitle kodlanabileceği hesaplanır. Entropi 3 bit, A algoritmasının ortalaması 5 bit ise artıklık $5 - 3 = 2$; B algoritmasının ortalaması 3,5 bit ise artıklık $3{,}5 - 3 = 0{,}5$ olur (ekrandan, 00:55:40). Artıklığı en küçük olan algoritma seçilir.
  - A'nın çıktısına bir C algoritması daha uygulanarak da artıklık düşürülebilir. Ama tek başına 0,5 artıklık veren bir yöntem varsa iki yöntemi arka arkaya kullanmak gereksiz olabilir; seçim önceliğe (yer, zaman ya da ikisi) bağlıdır.
- Veri sıkıştırmada kullanılan kavramlar (ekrandan, 01:00:00)
  - Compressor / encoder / kodlayıcı: input stream ham veriyi sıkıştırıp output stream oluşturan algoritma.
  - Decompressor / decoder / kod çözücü: kodlayıcının yaptığı işi ters yönde yapar. Her zaman birebir aynı adımların tersi uygulanmaz.
  - Codec: encoder ve decoder'ı birlikte tanımlar; blok şemalarda tek kutu olarak çizilir.
  - Unencoded / raw data / original data: orijinal giriş verisi. Encoded / compressed: sıkıştırılmış veri.
- Adaptif olmayan, adaptif ve yarı adaptif sıkıştırma (ekrandan, 01:20:00)
  - Non-adaptive: parametre ve işlemler belli tip veriler için geçerlidir (facsimile compression). Kodlar önceden, büyük bir derlem üzerinden belirlenir ve gelen veriye bakılmadan hep aynı kod kullanılır (ASCII tablosu gibi).
  - Adaptive: işlenmemiş her türlü veri üzerinde işlem yapar; parametre ve işlemler değiştirilebilir. Aynı sembolün kodu dokümanın farklı yerlerinde farklı olabilir.
  - Semi-adaptive: iki adımlıdır. Önce veri üzerinden geçilerek tüm sembollerin kullanım sıklıkları çıkarılır, sonra kodlama ve sıkıştırma yapılır.
  - "ababc" örneği (ekrandan, 01:23:40): yarı adaptif yöntem önce 2 a, 2 b, 1 c sayar; olasılıklar $2/5 = 0{,}4$, $0{,}4$ ve $1/5 = 0{,}2$ olur, kodlar buna göre atanır. Adaptif yöntem ise ilk "a" geldiğinde tek sembol gördüğü için onu kısa kodlar; "b" ve ikinci "a" geldikçe olasılıklar değişir ve semboller farklı uzunlukta kodlarla kodlanır.
  - Sıkıştırma başarısı en iyi olan adaptif, kodlaması en zor olan da adaptiftir; en kolay ve başarısı en düşük olan non-adaptiftir. Hoca örnek olarak adaptifin %70'ler, non-adaptifin %40 civarı sıkıştırma verebileceğini söyledi.
- Faks sıkıştırması
  - Faks, kâğıttaki metni de doküman olarak değil görüntü olarak algılar (siyah-beyaz, gri seviye ya da renkli). Görüntünün satırlarına scan line denir; bir karakterin yüksekliği birkaç scan line'a karşılık gelir.
- Kayıplı ve kayıpsız sıkıştırma (ekrandan, 01:20:00 ve 01:28:40)
  - Lossy: sıkıştırılmış veri açıldığında orijinal veriye dönülemez; ses ve görüntü sıkıştırmada kullanılır. Ekrandaki çizim: $P \to C \to P_s \to DC \to P'$.
  - Lossless: açılan veri orijinal hâlindedir ($P \to C \to P_s \to DC \to P$); doküman sıkıştırmada kullanılır.
  - Tıbbi görüntüde kayıplı sıkıştırma doğru değildir: MR görüntüsündeki çok küçük bir kitle gürültü sayılıp atılırsa hasta sağlıklı görünebilir. Duvardaki true color bir resimde gözün ayırt edemediği renkler atılabilir.
  - Dokümanda kayıplı sıkıştırma harflerin değişmesine yol açar. İçeriği bozmayan durumlarda (fazla boşlukları tek boşluğa indirmek) kayıp kabul edilebilir; ama "bir milyon TL" içindeki sıfırların kaybolup "bin TL" olması kabul edilemez.
- Simetrik ve asimetrik sıkıştırma (ekrandan, 01:28:40)
  - Symmetrical: kompresör ve dekompresör aynı algoritmayı ters yönde kullanır. Ekrandaki çizim: $a \to b \to P_s \to b \to a$; bir aynalanma vardır.
  - Asymmetric: kompresör ve dekompresör farklı algoritma kullanır. Örneğin görüntüdeki float değerler integer'a çevrilip işlenirse açarken orijinal değerlere dönülemez. Asimetrik bir yöntem kayıpsız da olabilir; o zaman orijinal veri elde edilir ama adımlar aynı işlemlerin ters sırası değildir.
- Stream mode ve block mode (ekrandan, 01:40:00)
  - Stream mode: veri byte byte çekilir ve üzerinde işlem yapılır; hızlandırmak için önbellek (cache) kullanılabilir.
  - Block mode: veri bloklar hâlinde çekilir ve her blok üzerinde farklı algoritma çalıştırılabilir.
- Birimler: bpc ve bpp (ekrandan, 01:40:00)
  - Bits per character: sıkıştırılan her sembolün ortalama bit uzunluğu. "3 bpc", bir karakter için ortalama 3 bit gerektiği demektir.

  - Bits per pixel: sıkıştırılan her pikselin ortalama bit uzunluğu. Siyah-beyaz resimde piksel 1 bittir (0 siyah, 1 beyaz); 8 renk için 2^3, yani 3 bit; true color resim 24 bittir. Sıkıştırma sonucunda bir piksel ortalama 12 bitle gösterilirse bpp 12 olur.
- Sıkıştırma oranı ve performansı (ekrandan, 01:44:20)
  - compression_ratio $= \dfrac{\text{size of the output stream}}{\text{size of the input stream}}$.
  - Sıkıştırma performansı (yüzdesel) $= 100 \cdot (1 - \text{compression_ratio})$.
  - Örnek: 100 birimlik veri 40 birime inerse oran $40/100 = 0{,}4$, performans $(1 - 0{,}4) \cdot 100 = \%60$.
- Derlem (corpus) (ekrandan, 01:46:00)
  - Yeni bir yöntemin başarısını göstermek için herkesin kullandığı standart veri setleridir: Calgary, Canterbury, UCI. Canterbury daha sonra çıkmıştır ve Java kodu, XML gibi farklı biçimde dosyalar da içerir.
  - Kendi seçilen veri setiyle elde edilen sonuç kimseyi inandırmaz; karşılaştırma ortak bir taban üzerinde yapılmalıdır.
- Görüntüde performans ölçütleri (ekrandan, 01:50:00 ve 01:55:20)
  - $I(r, c)$ orijinal resim, $I'(r, c)$ açılmış (decompress edilmiş) resim; error$(r, c) = I'(r, c) - I(r, c)$. Kayıplı sıkıştırmada resmin boyutu değişmez, kullanılan renk sayısı azalır (örneğin 100 renkten 60 renge).
  - Toplam hata, e_{RMS}, SNR_{RMS} ve $PSNR$ formülleri Detaylı Açıklamalar bölümündedir.
  - e_{RMS} ne kadar küçük, SNR ne kadar büyükse sonuç o kadar iyidir; ikisi ters orantılıdır. Yayınlarda kimi e_{RMS}, kimi SNR, kimi PSNR verir.

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritma seçimi uygulamaya göre değişir
  - Kimi uygulamada diskte kaplanan yer, kimisinde iletim hızı önemlidir; kimisinde sıkıştırmanın uzun sürmesi sorun değildir ama açmanın hızlı olması gerekir, kimisinde tersi. Bazı algoritmalar resimde, bazıları metinde daha başarılıdır. Seçim deneyimle ve algoritmaların nasıl çalıştığını kavrayarak yapılır.
- Artıklık "önemli bir şey"
  - Arka arkaya sıkıştırarak veriyi tek bite indirmek mümkün değildir; tek bitten orijinal veriye ulaşılamaz. Artıklığın en küçük olduğu algoritma seçilir.
- Sembollerin türü fark etmez
  - "a" ile yıldız, boşluk ile eğik çizgi aynıdır; hepsi birer semboldür. İstatistiksel yöntemlerde önemli olan sembolün ne olduğu değil, ne sıklıkla geçtiğidir.
- Kayıplı ve kayıpsız ayrımı
  - Ne zaman kayıplı, ne zaman kayıpsız algoritma seçileceği çok önemlidir: metin ve tıbbi görüntü kayıpsız, gündelik görüntü ve ses kayıplı sıkıştırılabilir.
- Sıkıştırma oranı yüzde değildir
  - Oran $0{,}4$ ise kazanç %60'tır; kazancı görmek için oranın 1'den farkı alınmalıdır.
- Sonuç raporlarken formül verilmeli
  - Hoca, karşılaştığı bir yayında sıkıştırma performansının yüzdesinin farklı bir tanımla verildiği için yanlış yorumlandığını anlattı. Hangi ölçütün kullanıldığı, gerekiyorsa formülüyle, açıkça yazılmalıdır.
- Hatanın karesi
  - $5 - 4 = 1$ ve $4 - 5 = -1$; farklar doğrudan toplanırsa $1 + (-1) = 0$ olur ve hata yokmuş gibi görünür. Karesi alınınca ikisi de 1 olur.

Kısa Tekrar Notları

- Sıkıştırma: input stream → (yer ve zaman kazancı için) → daha küçük output stream; açınca orijinale dönülür.
- Fixed size code: tüm semboller aynı bit (ASCII 8 bit). Variable size code: sık sembol az bit, seyrek sembol çok bit.
- Artıklık = algoritmanın ortalama bit uzunluğu − entropi. Örnek: $5 - 3 = 2$, $3{,}5 - 3 = 0{,}5$; küçük olan seçilir.
- Encoder/compressor/kodlayıcı; decoder/decompressor/kod çözücü; codec = ikisi birlikte.

- Non-adaptive: kodlar önceden ve sabit (faks). Adaptive: kodlar akış boyunca değişir. Semi-adaptive: önce frekans, sonra kod.
- "ababc": $P(a) = P(b) = 0{,}4$, $P(c) = 0{,}2$.
- Adaptif: en iyi sıkıştırma, en zor kodlama. Non-adaptif: en kolay, en düşük başarı.
- Lossy: açılan veri $P' \neq P$ (ses, görüntü). Lossless: açılan veri P'nin aynısı (doküman, tıbbi görüntü).
- Simetrik: aynı algoritma ters yönde ($a \to b \to P_s \to b \to a$). Asimetrik: farklı algoritmalar.
- Kodlayıcı yavaş, çözücü hızlı: arşiv. Kodlayıcı hızlı, çözücü yavaş: yedekleme (backup).
- Stream mode: byte byte. Block mode: bloklar, her bloğa farklı algoritma.
- bpc: karakter başına ortalama bit; bpp: piksel başına ortalama bit.
- Oran = çıktı/girdi; performans = $100 \cdot (1 - \text{oran})$. $40/100 = 0{,}4 \Rightarrow \%60$.
- Derlemler: Calgary, Canterbury, UCI.
- Görüntü: e_{RMS} küçük, SNR/PSNR büyük olmalı.

Detaylı Açıklamalar

- **Artıklığın sayısal yorumu.** Hoca artıklığı entropi ile bağlayarak anlattı (ekrandan, 00:55:40). Entropi henüz tanımlanmadı ama şunu söyler: bu veriyi sıkıştırmak için bir sembol ortalama 3 bitle kodlanmalıdır. Tek tek semboller 1, 2, 4 ya da 5 bit olabilir; ortalama 3'tür. A yöntemiyle sıkıştırıldığında semboller ortalama 5 bit çıkarsa fark $5 - 3 = 2$ olur ve bu büyük bir artıklıktır. B yöntemi ortalama 3,5 bit verirse artıklık $3{,}5 - 3 = 0{,}5$'tir; B seçilir. A'nın çıktısına C eklenerek de artıklık düşürülebilir, ama zaman da önemli olduğundan, tek başına yeterli bir yöntem varken iki yöntem arka arkaya çalıştırılmaz. Yine de bazı algoritmalar o kadar karmaşıktır ki iki basit algoritmayı art arda çalıştırmaktan daha uzun sürebilir; karar önceliğe göre verilir.
- **Üç kodlama türünün "ababc" örneği.** Hoca önce slayta "abab", sonra "ababc" yazdı (ekrandan, 01:21:40 ve 01:23:40).
  - Non-adaptif yöntem dokümana hiç bakmaz. Örneğin 10 milyon A4 sayfalık bir Türkçe doküman topluluğunda "a" ve "b"nin kaç kez geçtiğinden olasılıkları bulur, kodu bir kez atar ve bundan sonra gelen her veride aynı kodu kullanır. Bugün ekranda gördüğümüz metin ASCII ile, yani önceden hazırlanmış aynı uzunlukta kodlarla yazılmıştır.
  - Semi-adaptif yöntem önce dokümanı sayar: 2 a, 2 b, 1 c. Olasılıklar $2/5 = 0{,}4$, $2/5 = 0{,}4$, $1/5 = 0{,}2$'dir. Bu olasılıklara göre kodları atar ve dokümanı sıkıştırır.
  - Adaptif yöntemde önce "a" gelir; o ana kadar görülen her şey "a" olduğu için olasılığı %100'dür ve hoca bunu 1 bitle kodladığını söyledi. Sonra "b" gelir ve belki 2 bitle kodlanır. Sonra yine "a" gelir; üç karakterden ikisi "a" olduğu için olasılığı $2/3$ olur ve kodu değişir. Dokümanın sonuna kadar her sembolün kodu defalarca değişebilir. Hocanın söylediği bit sayıları açıklama içindir; gerçek uzunlukları kullanılan algoritma belirler.
- **Asimetrik sıkıştırmanın uygulamaları.** Kodlayıcının karışık ve uzun, çözücünün basit ve hızlı olduğu durum arşiv uygulamalarıdır: veri arşive bir kez konur, ama gün içinde farklı kişiler tarafından sürekli indirilir. Tersi, kodlayıcının basit ve hızlı, çözücünün karışık ve yavaş olduğu durum yedekleme (backup) uygulamalarıdır: bankalar her akşam gün sonu yedeği alır ve bu işlemin kısa sürmesi gerekir; eski veriye dönüş ise belki yılda bir kez gerekir. Hoca bugünün donanımlarında bu sürelerin saatler değil, 10 saniyeye karşı 20 saniye gibi farklar olduğunu ekledi.
- **Görüntüde performans formülleri.** Ekrandaki tanımlar şunlardır ($M \times N$ boyutlu resim; ekrandan, 01:50:00):

$$\text{error}(r, c) = I'(r, c) - I(r, c)$$

$$\text{total error} = \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} [I'(r, c) - I(r, c)]$$

$$e_{RMS} = \sqrt{\frac{1}{N^2} \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} \left[I'(r,c) - I(r,c)\right]^2}$$

$$SNR_{RMS} = \sqrt{\frac{\sum_{r=0}^{M-1} \sum_{c=0}^{N-1} I'(r,c)^2}{\sum_{r=0}^{M-1} \sum_{c=0}^{N-1} \left[I'(r,c) - I(r,c)\right]^2}}$$

$$PSNR = 10 \log_{10} \frac{(L-1)^2}{\frac{1}{N^2} \sum_{r=0}^{M-1} \sum_{c=0}^{N-1} \left[I'(r,c) - I(r,c)\right]^2}$$

Burada L renk sayısıdır; renkler 0'dan başladığı için $L-1$ alınır. Hoca N^2'nin aslında $N \cdot M$ olduğunu, formülü basitleştirmek için kare resim varsayıldığını ve gerekirse padding ile resmin kare yapılabileceğini söyledi. PSNR logaritmik ölçek kullandığı için sonuçları göstermekte rahattır; SNR ile aynı yorumu verir.

  - Ders sonrası not: Ekrandaki SNR_{RMS} formülünün paydasında farklar kare alınmadan yazılmıştı; hoca da anlatımda farkların karesinin alındığını söyledi. Yukarıda paydada kareli biçim yazıldı. Ekrandaki PSNR ve SNR formüllerinde iç toplamın üst sınırı "$R-1$" olarak geçiyordu; yukarıda $N-1$ olarak yazıldı.
  - Ders sonrası not: Hoca e_{RMS} küçükse "redundansı az, buna daha fazla sıkıştırma uygulayamazsın" dedi. e_{RMS} ve SNR büyük olasılıkla artıklığı değil, açılan görüntünün orijinale ne kadar yakın olduğunu (yeniden oluşturma kalitesini) ölçer.

- **Derlemler.** Hoca Calgary ve Canterbury derlemlerini Kanada'daki üniversitelerin geliştirdiğini, UCI'nin ise Kaliforniya'daki bir üniversiteye ait olduğunu ve genelde görüntü işlemeyle ilgili olduğunu söyledi.
  - Ders sonrası not: Canterbury derlemi büyük olasılıkla Yeni Zelanda'daki University of Canterbury'de hazırlanmıştır. UCI büyük olasılıkla University of California, Irvine'dır; bu kurumun bilinen veri deposu yalnız görüntüyle sınırlı değildir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.